



2026

Reverse Engineering

ACT - NSW - NT - QLD - SA - TAS - VIC - WA

**Ngunnawal & Ngambri - Dharawal, Yuin & Wadi Wadi - Larrakia
Turrbal & Yuggera - Kaurna - Palawa & Pukana - Wurundjeri - Noongar**



2026

PECAN+ Capture The Flag 2026

A national cyber security competition for Australian high school-aged competitors.

August 22nd & 24rd, 2026

ACT - NSW - NT - QLD - SA - TAS - VIC - WA

**Ngunnawal & Ngambri - Dharawal, Yuin & Wadi Wadi - Larrakia
Turrbal & Yuggera - Kurna - Palawa & Pukana - Wurundjeri - Noongar**

Overview

1. Introduction
2. Computers Hardware
3. Computer Programs
4. Techniques
5. Binary Patching (A)
6. Binary Exploitations (A)
7. Example Challenges
8. Conclusion

Introduction

- 'binary' -> a **compiled program** ('.exe', '.dll', '.elf', '.so', etc.)
- Reverse Engineering is the act of **analysing** a compiled **binary** to find out **what it does/how it does it**



Reasons to Reverse Engineer

1. Discover Vulnerabilities

- For CTFing
- To discover real vulnerabilities

2. Patch Programs

- Debugging
- Video Game Modification

3. Analyse malware

- Find out what it did
- Find out how to stop/break it



2026

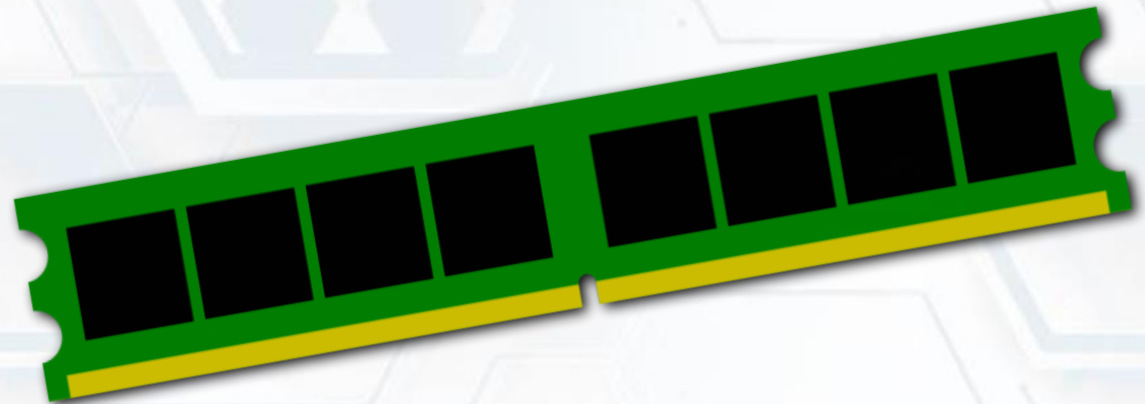
Computer Hardware

ACT - NSW - NT - QLD - SA - TAS - VIC - WA

**Ngunnawal & Ngambri - Dharawal, Yuin & Wadi Wadi - Larrakia
Turrbal & Yuggera - Kurna - Palawa & Pukana - Wurundjeri - Noongar**

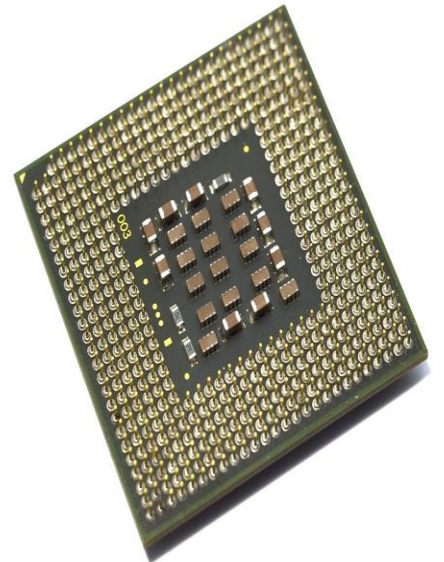
Memory

- Stores the **instructions** and **data** for the computer to carry out
- Each location is accessed by a numerical **memory address** in hexadecimal e.g. 0xdeadbeef
- This is your RAM



CPU

- Performs the **instructions** that the program provides it
- Has an **instruction set** which lists all the binary instructions it understands
- Has **registers** which are special memory locations housed on the CPU that store and manipulate data for the instructions
 - RAX – Accumulator (math operations get output here)
 - RSP – Stack Pointer (Points to the top of the stack)
 - RIP – Instruction Pointer (Points to the next instruction for the CPU to read)





2026

Computer Programs

ACT - NSW - NT - QLD - SA - TAS - VIC - WA

**Ngunnawal & Ngambri - Dharawal, Yuin & Wadi Wadi - Larrakia
Turrbal & Yuggera - Kurna - Palawa & Pukana - Wurundjeri - Noongar**

Computer Programs

- A computer program is a **sequence of instructions** which tell the computer how to behave
- A program can be represented in **text** or **binary**, and the same program can be written multiple ways
- On Windows they are commonly **'`.exe`'** or **'`.dll`'**
- On Linux they are **'`.elf`'** or **'`.so`'**

Source code

- Source code is a textual program written in a **programming language**
- Easily **understandable** by **humans**
- Tells the CPU the **high-level** of what to do

```
1  #include <stdio.h>
2
3  int main() {
4      printf("Hello World!\n");
5      return 0;
6  }
```


Assembly Code

- Source code that resembles the **CPU instructions**
- **Understandable** by humans after some learning
- Tells the CPU **exactly** what to do
- Different CPUs have different assembly

```
1  .file "hello.c"
2  .text
3  .section .rodata
4  .LC0:
5  .string "Hello World!"
6  .text
7  .globl main
8  .type main, @function
9  main:
10 .LFB0:
11 .cfi_startproc
12     endbr64
13     pushq   %rbp
14     .cfi_def_cfa_offset 16
15     .cfi_offset 6, -16
16     movq    %rsp, %rbp
17     .cfi_def_cfa_register 6
18     leaq    .LC0(%rip), %rax
19     movq    %rax, %rdi
20     call    puts@PLT
21     movl    $0, %eax
22     popq    %rbp
23     .cfi_def_cfa 7, 8
24     ret
25     .cfi_endproc
26 .LFE0:
27 .size main, .-main
28 .ident "GCC: (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0"
29 .section .note.GNU-stack,"",@progbits
30 .section .note.gnu.property,"a"
31 .align 8
32 .long 1f - 0f
33 .long 4f - 1f
34 .long 5
35 0:
36 .string "GNU"
37 1:
38 .align 8
39 .long 0xc0000002
40 .long 3f - 2f
41 2:
42 .long 0x3
43 3:
44 .align 8
45 4:
```


Transformations

Source Code → Binary

- **Compiling**
 - Source code → Assembly
- **Assembling**
 - Assembly → Machine code

Binary → Source Code

- **Disassembling**
 - Machine code → Assembly
- **Decompiling**
 - Assembly → Source code

Executable File Formats

- **Windows**

- '.exe' (a standalone executable)
- '.dll' (Dynamically Loaded Library that a '.exe' imports)

- **Linux**

- '.elf' (Executable and Linkable Format, a standalone executable)
- '.so' (Shared Object, a library '.elf's load)

- **MacOS**

- '.app' (a standalone executable, contains the libraries imported)

Executable and Linkable Format (ELF)

- Machine code format for **Linux**
- **Linkable** means it can be combined with other ELF files to create a bigger ELF file
- Made of **Symbols, Sections, and Segments**
- **Symbols**
 - A named memory address that stores a value
- **Sections**
 - Contains the instructions, data, and metadata
- **Segments**
 - Contains multiple 'Sections' that can be executed

Sections

- There are lots of different sections in ELF files
- Three main ones are:
 - **.text**
 - Holds the executable instructions for the program/the assembly
 - **.bss**
 - Holds uninitialised data
 - **.data**
 - Holds initialised data

How Programs use Memory

- The running memory typically in **RAM**
- There are two data structures used, a 'heap' and a 'stack'
- **Heap**
 - A large pool of memory that is dynamically allocated (e.g. user input of unknown length)
 - Persists for the entire length of the program, unless it is told to remove it (it just pretends the data is no longer there)
- **Stack**
 - A Last In First Out (LIFO) data structure
 - Stores local variables
 - Only data of pre-known sizes

How the Stack is used

- Functions exist in locations at memory
- When one function called another function, the program needs to know where to return to after the new function is done
- Therefore, add the address to return to the top of the stack
- The arguments passed into the function are then added to the stack
- Then any local variables created





2026

Techniques

ACT - NSW - NT - QLD - SA - TAS - VIC - WA

**Ngunnawal & Ngambri - Dharawal, Yuin & Wadi Wadi - Larrakia
Turrbal & Yuggera - Kurna - Palawa & Pukana - Wurundjeri - Noongar**

Techniques

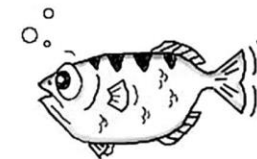
- In general, we try to find the **desired program state** typically where the flag is **in memory**, or **displayed**
- The main techniques split for Reverse Engineering split into two categories
- **Static Analysis**
 - Analysing the program by looking at it and trying to figure out what it does
- **Dynamic Analysis**
 - Analysing the program by running it and seeing what it does

Static Analysis

- Analysing the program **without running it**
- Speculate on what it might do/how it works
- This is typically done by either looking at a **decompiled/disassembled** version or analysing the compiled binary

Static Analysis Tools

- **Disassembler**
 - objdump, GNU General Debugger (GDB)
- **Decompiler**
 - Ghidra, dotPeek, Radare2, IDA
- **Miscellaneous**
 - file, strings, grep, readelf, nm



GDB
The GNU Project
Debugger

Ghidra

- A Reverse Engineering tool that **disassembles** and **decompiles**
- The one I recommend as it is free, and open source



Analysing 'static'

1. Open it up in 'ghidra'
2. What does the program do?
3. After statically analysing it, can you make it print the flag?
4. Look at the source code, are they the same?
5. If they are different why are they different?
6. What does 'strings' do?

Binary Patching

- If the program has checks that are awkward to get around, or certain instructions they run that are harmful to the computer (fork bomb, etc.)
- 'Binary Patching' is the process of editing the assembly and reassembling so the binary now acts differently
- It can be done easily in Ghidra
- It does require you know assembly

Binary Patching 'patch'

1. I go through patching 'walkthrough'
2. Try to use Ghidra to patch the 'patch' binary to just always print the flag

Dynamic Analysis

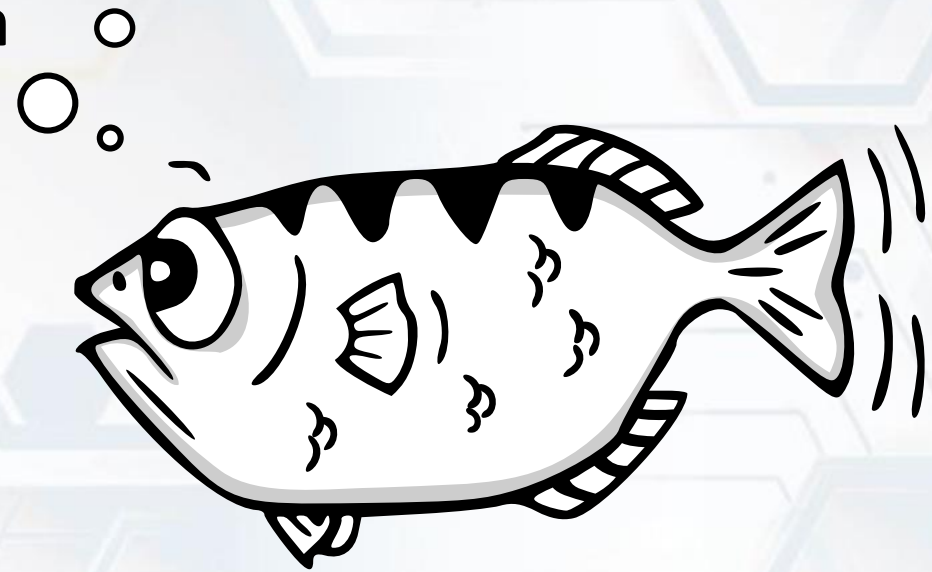
- Analyse the program by **executing it**
- Much **easier** and **faster** to do then static analysis
- Let's you **view** the **memory** at runtime and do **breakpoint debugging**
- A **bad idea** to do on your system
- If you run **malware** on your system and then get **infected**, that's **not my fault**
- We do it on VMs or special machines just for this

Dynamic Analysis Tools

- **Debugger**
 - GDB, LLDB, OllyDGB
- **Emulator**
 - Qemu
- **Execution Tracing**
 - ltrace, strace

GDB

- GDB allows for **interactive debugging** of binary programs and the **inspection** of its state.
- It can:
 - **Start** a program with custom conditions.
 - Make a program **stop** on specified conditions.
 - **Examine** what has happened when a program has stopped.
 - **Change** things in a program.



GDB Usage

- GDB is a command which is invoked with the program to be debugged as its argument.
 - It presents a prompt which accept various commands.
- **Commands**
 - run
 - break
 - continue
 - disassemble
 - info registers
 - print
 - set
 - x
 - exit

Analyse 'dynamic'

- Run 'dynamic' with GDB
- Print address at 'main'
- Figure out address just after 'get_flag' is called
- Add break point
- Run till breakpoint
- Examine stack to get memory address of flag
- Examine heap to get memory address



2026

Binary Exploitation (Pwn)

Binary Exploitation (Pwn)

- This is a more advanced version of RE, where instead of just figuring out how the program works, you figure out what **vulnerabilities** are in the software
- This is useful as while **RE controls binaries on your own device**, with **Pwn** you can control **binaries on other's devices**
- This is typically done through **overflows**, or **formatting vulnerabilities**

Overflows (Pwn)

- This is when the **data** in a fixed length variable is of a **greater** size **then it should be** and the data 'overflows'
- For **integers/floats**, this leads to numbers looping from one extreme to another e.g. (' $2^{32} - 1$ ' to '0') or (' $2^{31} - 1$ ' to ' $-2^{31} + 1$ ')
- For **arrays (strings)**, this leads to the attacker writing data into the memory after the string, this is called a **buffer overflow**, and breaks up into **heap overflows**, and **stack overflows**

Buffer Overflows (Pwn)

- These allow the user to **arbitrarily write to memory**, and thereby **change the values of variables**
- Heap Overflows
 - An attacker can **change the global variables** in memory
- Stack Overflows
 - An attacker can **change local variables** located below in the stack
 - An attacker can change the 'return address' variable to **run arbitrary functions**
 - If a 'command' then 'return' can be found (called **gadgets**), then these can be chained together to **run arbitrary code** if the stack is large enough

Echo Chamber

- Use Ghidra to analyse the binary
- Do a buffer overflow to get the flag printed out



2026

Example Challenges

Example Challenges

- Let's reverse engineer a few binary programs using different techniques.
- Each program will have a `check_flag` function.
- You'll need Ghidra, and GDB.
- <https://github.com/revengctf/crackmes>

Conclusion

- Hopefully you learnt something
- This is a **very big topic**, and has a lot of needed knowledge
- If you are doing **Dynamic Analysis** on an **unknown binary**, be **careful**
- If you install malware and infect your machine that is **not** my fault
- **No Crime**, if you hack something or break copyright, make sure you know the laws first (consult a lawyer) (and don't say I taught you)

Thank you to our PECAN+ CTF 2026 Sponsors



2026

Platinum Sponsor:



Australian Government
Australian Signals Directorate

Platform Sponsor:



Gold Sponsors:



Silver Sponsors:

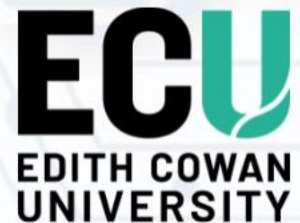


Department of the Premier and Cabinet
Office of Digital Government



And all local Sponsors around Australia.

PECAN+ is an initiative
that is proudly led by
Edith Cowan University



Thank you to national PECAN+ CTF 2026 hosts

PECAN+ CTF is a collaborative initiative, proudly led by Edith Cowan University, and brought to life through the support of Government agencies, higher education & vocational institutions and cyber industry around Australia.

